

Main page

Big Data Pipeline Monitor je aplikace pro evidenci, spouštění a sledování datových pipeline nad datovými zdroji, včetně monitoringu běhů a upozornění na problémy.

V tomto školním projektu tedy jde o simulaci orchestrace a monitoringu, ne o plnou data platformu.

Cílem projektu je navrhnout a implementovat systém pro monitorování datových pipeline.

Systém eviduje:

- datové zdroje (datasets)
- datové pipeline
- běhy pipeline
- kroky pipeline
- alerty

Projekt simuluje reálné systémy používané v datových platformách, například:

- Apache Airflow
- Databricks Jobs
- AWS Glue

Studenti implementují:

- backend architekturu (tutorial 2)
- frontend aplikaci (tutorial 3)

Aplikace dělá

- eviduje datasety
- eviduje pipeline
- eviduje běhy pipeline
- zobrazuje stav běhů
- umožňuje pipeline spustit
- umožňuje sledovat alerty
- poskytuje přehled a monitoring

Aplikace nedělá

- neprovádí skutečné distribuované výpočty
- není to Spark ani Airflow
- neřeší fyzickou orchestraci clusteru
- nemusí umět skutečný cron scheduler
- nemusí pracovat s reálnými big data soubory

Minimální výstupy

Backend minimum

- validace vstupu
- error handling
- oddělení business logiky
- endpointy:
 - POST /datasets
 - GET /datasets
 - GET /datasets/:id
 - POST /pipelines
 - GET /pipelines
 - GET /pipelines/:id
 - POST /pipelines/:id/run
 - GET /runs
 - GET /runs/:id
 - PATCH /runs/:id (pro změnu stavu)
 - POST /alert-rules
 - GET /alert-rules
 - GET /alert-rules/:id
 - GET /alerts
 - GET /alerts/:id
 - PATCH /alert-rules/:id (volitelně)
 - DELETE /alert-rules/:id (volitelně)

Frontend minimum

- seznam pipeline
- detail pipeline
- seznam runů
- akci „Run pipeline“
- základní dashboard nebo summary panel
- loading / empty / error stavy

Hlavní scénáře použití

Založení datasetu

Správce nebo admin založí dataset.

Například:

- název: customer_transactions
- popis: transakce zákazníků z e-shopu
- owner: analytics-team

Tím vznikne evidovaný datový zdroj.

Aplikace zatím neřeší, kde fyzicky data leží. Pro výuku stačí evidovat metadata.

Založení pipeline

Admin založí pipeline nad existujícím datasetem.

Například:

- dataset: customer_transactions
- pipeline: daily-aggregation
- schedule: každý den ve 2:00
- active: ano

Tím vznikne definice procesu, který se má pravidelně spouštět.

Důležité je, že pipeline musí odkazovat na existující dataset.

Zobrazení seznamu pipeline

Uživatel otevře přehled pipeline a vidí například:

- název pipeline
- vstupní dataset
- aktivní/neaktivní stav
- plán spuštění
- poslední run
- poslední status

To je typicky hlavní dashboard.

Manuální spuštění pipeline

Operator klikne na „Run pipeline“.

Systém:

- ověří, že pipeline existuje
- ověří, že je aktivní
- založí nový JobRun
- nastaví stav running
- uloží startedAt

Pipeline nemusí opravdu Spark job, **stačí simulovat spuštění a následný výsledek**.

Například:

- při startu vznikne run ve stavu running
- po čase nebo jinou operací se přepne na `success` nebo `failed`

Monitoring běhu

Uživatel si otevře detail pipeline nebo detail runu.

Vidí:

- status
- čas spuštění
- čas dokončení
- délku běhu
- počet zpracovaných záznamů
- případně chybovou hlášku

V případě modelování i kroků běhu, je možné vidět i:

- extract – success
- transform – running
- load – pending

Tady je hlavní hodnota aplikace: **operativní monitoring**.

Vznik alertu

Když se stane problém, aplikace vytvoří alert.

Například:

- run skončí failed
- runtime překročí limit
- pipeline nebyla spuštěna podle plánu

Alert se objeví v seznamu alertů.

Operator pak může zjistit:

- co se stalo
- kterého běhu se problém týká
- která pipeline je postižená

Hlavní části UI

1. Dashboard

Hlavní přehled systému.

Může zobrazovat:

- počet datasetů
- počet pipeline
- počet aktivních pipeline
- počet běhů za poslední období
- počet failed runů
- počet otevřených alertů

To dává uživateli rychlý přehled o stavu platformy.

2. Seznam datasetů

Tabulka datasetů:

- name
- owner
- schemaVersion
- createdAt

Akce:

- detail datasetu
- případně vytvořit nový dataset

3. Seznam pipeline

Tabulka pipeline:

- name
- dataset
- schedule
- active
- poslední run
- poslední status

Akce:

- detail pipeline
- run pipeline
- aktivovat / deaktivovat

4. Detail pipeline



Obsahuje:

- základní metadata pipeline
- odkaz na dataset
- schedule
- aktivní stav
- seznam běhů
- přehled alertů
- tlačítko na spuštění

Může obsahovat i statistiku:

- počet úspěšných běhů
- počet failed běhů
- průměrnou dobu běhu

5. Seznam běhů

Tabulka runů:

- pipeline
- status
- startedAt
- finishedAt
- runtime
- recordsProcessed

Filtry:

- podle pipeline
- podle statusu
- podle data

6. Detail běhu

Detail jednoho runu:

- pipeline
- verze pipeline
- status
- start
- finish
- error message
- records processed

Pokud je použit JobRunStep, pak i:

- seznam kroků
- stav každého kroku

7. Seznam alertů

Tabulka alertů:

- pipeline
- run
- message
- createdAt
- severity
- stav alertu

Logika backendu

1. Vytvoření pipeline

Uživatel pošle request na vytvoření pipeline.

Backend musí:

1. zvalidovat vstup
2. ověřit, že dataset existuje
3. uložit pipeline
4. vrátit vytvořený objekt

- Zde je použito business pravidlo:
- pipeline nemůže vzniknout nad neexistujícím datasetem

2. Spuštění pipeline

Uživatel pošle požadavek na spuštění pipeline.

Backend musí:

1. načíst pipeline
2. ověřit, že existuje
3. ověřit, že je aktivní
4. založit `JobRun`
5. nastavit status `running`
6. vrátit `run`

Volitelně může:

- vygenerovat kroky `runu`
- simulovat doběhnutí
- vytvořit alert při `failu`

3. Ukončení běhu

V případě této školní úlohy slouží k nastavení stavu `run` do finálního stavu.

Například:

- `PATCH /runs/:id`
- `status -> success` nebo `failed`

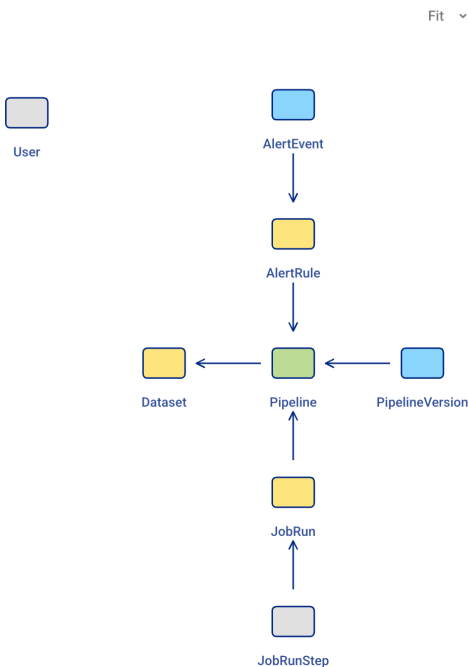
Pokud je `fail`:

- uložit `errorMessage`
- vytvořit `AlertEvent`

Typický životní cyklus pipeline

1. Dataset existuje.
2. Nad datasetem vznikne pipeline.
3. Pipeline je aktivní a má plán nebo je ručně spustitelná.
4. Spuštění pipeline vytvoří `JobRun`.
5. Run přechází stavy:
 - `pending`
 - `running`
 - `success / failed`
6. Pokud nastane problém, vznikne alert.
7. Uživatel sleduje běhy a řeší incidenty.

Doménový model



Doménová pravidla

- Pipeline lze spustit pouze pokud: `Pipeline.active == true`
- Pipeline může mít pouze jednu aktivní verzi: `PipelineVersion.active == true`
- Run může přejít pouze
 - `pending` → `running`
 - `running` → `success`
 - `running` → `failed`
- Alert se vyvolá pokud `runtime > threshold` nebo `status == failed`

Typické use cases

- Vytvoření datasetu
`POST /datasets`
- Vytvoření pipeline - pipeline musí referencovat existující dataset
`POST /pipelines`
- Spuštění pipeline - vytvoří `JobRun`
`POST /pipelines/:id/run`
- Monitoring běhů
`GET /runs`

Zjednodušení pro výuku

- datasety jsou jen metadata, ne reálné soubory
- pipeline je konfigurační objekt
- run je simulovaný běh
- alert vzniká z jednoduchého pravidla
- cílem není big data engine, ale architektura aplikace

Dataset

Dataset představuje zdroj dat.

Dataset je **nezávislá entita**. Pipeline dataset pouze **zpracovává**. Na jeden dataset může existovat více pipeline.

Například:

- transactions
- customer_events
- logs

```
1 {
2   "id": "uuid",
3   "name": "customer-transactions",
4   "description": "Raw transaction dataset",
5   "owner": "data-team",
6   "schemaVersion": 3,
7   "createdAt": "timestamp",
8   "updatedAt": "timestamp"
9 }
```

PipelineVersion

Pipeline může mít více verzí, např.

- změna SQL
- změna Spark jobu
- změna parametrů

```
1 {
2   "id": "uuid",
3   "pipelineId": "uuid",
4   "version": 3,
5   "config": {
6     "engine": "spark",
7     "query": "SELECT ..."
8   },
9   "createdAt": "timestamp"
10 }
```

JobRunStep

Velké pipeline mají více kroků. Například

- extract
- transform
- load

```
1 {
2   "id": "uuid",
3   "runId": "uuid",
4   "name": "transform",
5   "status": "running",
6   "startedAt": "timestamp",
7   "finishedAt": null
8 }
```

AlertEvent

AlertEvent vzniká při splnění podmínky alertu.

```
1 {
2   "id": "uuid",
3   "ruleId": "uuid",
4   "runId": "uuid",
5   "message": "Pipeline runtime exceeded threshold",
6   "createdAt": "timestamp"
7 }
```

Pipeline

Pipeline představuje proces zpracování dat.

Například:

- denní agregace
- detekce podvodů
- feature engineering

Pipeline vždy pracuje nad datasetem.

```
1 {
2   "id": "uuid",
3   "datasetId": "uuid",
4   "name": "daily-aggregation",
5   "description": "Daily revenue aggregation",
6   "schedule": "0 2 * * *",
7   "active": true,
8   "createdAt": "timestamp"
9 }
```

JobRun

JobRun reprezentuje **konkrétní běh pipeline**.

Status:

- pending
- running
- success
- failed

```
1 {
2   "id": "uuid",
3   "pipelineId": "uuid",
4   "pipelineVersion": 3,
5   "status": "running",
6   "startedAt": "timestamp",
7   "finishedAt": null,
8   "recordsProcessed": 0,
9   "errorMessage": null
10 }
```

AlertRule

Alert definuje podmínku, kdy se má vyvolat upozornění.

```
1 {
2   "id": "uuid",
3   "pipelineId": "uuid",
4   "name": "runtime alert",
5   "condition": "runtime > 10m",
6   "enabled": true,
7   "createdAt": "timestamp"
8 }
```

User

Pro účely autorizace.

Role:

- admin
- operator
- viewer

```
1 {
2   "id": "uuid",
3   "email": "user@company.com",
4   "role": "admin"
5 }
```